UTN FRM · TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN · BASES DE DATOS II · PROF. SERGIO NEIRA

# Informe post-incidente

Mantenimiento, respaldo y recuperación de Food Store — TP Unidad 6 (versión corregida)

**Grupo 4:** Belén Calvo · Hernán González · Elías Tello · Bruno Fiouchetta **Ensayos:** 28 al 30/10/2026 (grupo) y 07/11/2026 (corrección)
**Entorno:** PostgreSQL 17.11 · Windows 11 · Git Bash

**Repositorio:** https://github.com/belcalvo93/base-datos-2/tree/main/mantenimiento

| RTO OBSERVADO | ANTIGÜEDAD DEL RESPALDO | HUELLA DE REFERENCIA | INTEGRIDAD |
|---|---|---|---|
| **4,21 s** | **19 s** | **11/11** | **8/8 en 0** |
| Meta: ≤ 15 min | RPO de diseño: ≤ 24 h | Filas, importes y estructura | Valores y referencias huérfanas |

## 1 Objetivo y entorno

Demostrar que Food Store puede respaldarse, monitorearse, mantenerse y **recuperarse** con evidencia de cada etapa. La base principal `foodstore` (100 MB) es una copia de `bd2_tp3`, la base con la carga masiva del TP3: 6 tablas, 499.263 líneas de detalle y $629.621.805,75 facturados. Toda prueba destructiva se hizo sobre la base descartable `foodstore_restore_test`: `foodstore` nunca se borró ni se restauró.

Este informe corresponde a la versión corregida del 07/11. Los cuatro integrantes ya habían restaurado el respaldo en su PC entre el 28 y el 30/10 (sección 4). La corrección amplió las validaciones, agregó el registro PASS/FAIL con SHA256, la autorización escrita y el diagnóstico de VACUUM, y se volvió a ejecutar todo con los scripts finales.

## 2 Respaldo y su registro

`backup_foodstore.sh` sigue cuatro pasos y deja **una línea por ejecución** en `logs/backup_foodstore.log`:

- **Huella previa** con `conteo_referencia.sql`, guardada en `.conteo` junto al respaldo (sección 3).
- **Respaldo** con `pg_dump -Fc` y nombre con fecha y hora (`foodstore_AAAAMMDD_HHMMSS.dump`).
- **Verificación inmediata**: tamaño mayor a 0 y lectura del índice del archivo con `pg_restore -l`, para detectar un archivo dañado el día del respaldo y no durante un incidente.
- **SHA256** del `.dump`, guardado en `.sha256`. El simulacro lo vuelve a calcular antes de restaurar.

| FECHA Y HORA | RESULTADO | ARCHIVO | BYTES | SHA256 | TOC |
|---|---|---|---|---|---|
| 07/11 22:26:54 | PASS | `foodstore_20261107_222649.dump` | 9.428.332 | `a1e50a50…f52b3b` | 54 |
| 07/11 22:28:25 | FAIL | Prueba controlada con `base_inexistente`: frena en el paso `conteo`, sale con código 1 y no deja archivos a medias | | | |
| 07/11 22:29:01 | PASS | `foodstore_20261107_222858.dump` | 9.428.332 | `a4819d08…1d272e0d` | 54 |
| **07/11 23:11:54** | PASS | **`foodstore_20261107_231151.dump`** | 9.428.332 | `0192618b…bce1789a` | 54 |

El último es el respaldo del simulacro final. Su SHA256 completo es `0192618bca3a8f96c18f321b544ace2ef835652dd5bb5ede3d509883bce1789a`. Los `.dump` no se versionan porque incluyen la tabla `usuario` con hashes de contraseñas; su creación, tamaño, SHA256 y restauración constan en los logs.

## 3 Restauración y validaciones

`restore_test.sh` toma el último respaldo y, en este orden: verifica el SHA256, pide la **autorización escrita** para borrar la base descartable (sección 7), la recrea desde `template0`, confirma con `SELECT current_database()` que la conexión apunta a ella, restaura con `pg_restore --exit-on-error --no-owner --no-privileges` y corre las dos validaciones. Ya no consulta `foodstore`: compara contra la huella guardada al momento del respaldo, así una escritura posterior no produce un falso fallo.

### Validación 1 — Huella restaurada contra respaldo

| CLAVE | RESTAURADA | RESPALDO | |
|---|---|---|---|
| categoria | 5 | 5 | OK |
| cliente | 20.005 | 20.005 | OK |
| producto | 50.011 | 50.011 | OK |
| pedido | 200.005 | 200.005 | OK |
| detalle_pedido | 499.263 | 499.263 | OK |
| usuario | 3 | 3 | OK |
| **total_facturado** | 629.621.805,75 | 629.621.805,75 | OK |
| tablas / vistas | 6 / 5 | 6 / 5 | OK |
| restricciones / índices | 18 / 13 | 18 / 13 | OK |

### Validación 2 — Indicadores de integridad

| INDICADOR | RESULTADO |
|---|---|
| Precio negativo (`producto`) | 0 |
| Stock negativo (`producto`) | 0 |
| Cantidad no positiva (`detalle_pedido`) | 0 |
| Precio unitario negativo (`detalle_pedido`) | 0 |
| Detalle sin pedido | 0 |
| Detalle sin producto | 0 |
| Pedido sin cliente | 0 |
| Producto sin categoría | 0 |

**Sobre los subtotales:** en este esquema el subtotal no se almacena, se calcula como `cantidad × precio_unitario` (decisión de normalización del TP1). Por eso no puede haber "subtotales incorrectos" guardados; en su lugar se compara el **total facturado** (la suma de todos los subtotales) entre el

respaldo y la base restaurada. Si coincide, coinciden los importes, no solo la cantidad de filas. Las cuatro referencias huérfanas en 0 confirman además que las claves foráneas se restauraron, porque `pg_restore` carga los datos antes de crear las restricciones.

**Qué no recupera la restauración.** Se recuperan datos, estructura, vistas, restricciones e índices. **No se recuperan roles ni permisos**: `pg_dump` no incluye los objetos globales del clúster y la restauración usa `--no-owner --no-privileges`, así que los roles del TP5 (`app_web`, `admin_datos`, `rol_reportes_login`, etc.) y sus `GRANT` no vuelven. La propuesta está en la sección 8.

## 4 RPO y RTO: metas y valores observados

Las metas se fijaron en `SPEC_U6.md` antes de los ensayos: **RTO ≤ 15 min** y **RPO ≤ 24 h** (un respaldo diario). El RTO se mide desde inmediatamente antes de `createdb` hasta que terminan las validaciones. La antigüedad se mide desde la fecha del respaldo (tomada de su nombre) hasta el inicio de la restauración, sin contar la espera de la autorización.

| ENSAYO | SCRIPTS | RESPALDO USADO | CREATEDB + PG_RESTORE | RTO | ANTIGÜEDAD | RESULTADO |
|---|---|---|---|---|---|---|
| **Belén · 07/11 23:12 (final)** | finales | `foodstore_20261107_231151` | 3,53 s | **4,21 s** | **19 s** | PASS |
| Belén · 07/11 22:36 | corrección (huella de 10) | `foodstore_20261107_222858` | 5,27 s | 5,97 s | 435 s* | PASS |
| Belén · 28/10 | versión anterior | `foodstore_20261028_211058` | 2,53 s | 2,96 s | 66 s | OK |
| Hernán · 29/10 (PG 18.6) | versión anterior | `foodstore_20261029_162309` | 2,86 s | 3,20 s | 54 s | OK |
| Elías · 29/10 | versión anterior | `foodstore_20261029_201505` | 3,53 s | 3,84 s | 18 s | OK |
| Bruno · 30/10 | versión anterior | `foodstore_20261030_130826` | 3,82 s | 4,29 s | 15 s | OK |

* En ese ensayo la antigüedad incluía la espera de la autorización. Se corrigió a partir de la revisión de OpenCode (sección 7). Los ensayos con la versión anterior validaban 5 conteos de filas y 2 indicadores; el final valida la huella de 11 claves y 8 indicadores.

**Cómo leer el RPO.** La antigüedad observada (15 a 66 s) mide qué tan fresco era el respaldo, **no** datos perdidos: `foodstore` no recibió escrituras durante los ensayos. El RPO real lo fija la frecuencia del respaldo: con uno diario, el peor caso es perder hasta 24 h de cambios. Hoy el respaldo se ejecuta a mano, así que ese RPO depende de que alguien lo corra (mejora en la sección 8). El RTO, en cambio, sí es representativo: entre 3 y 6 s en cuatro PCs distintas, muy por debajo de la meta.

## 5 Mantenimiento justificado

`maintenance.sql` compara cada tabla contra los umbrales de autovacuum de la configuración (`autovacuum_analyze_threshold = 50`, `scale_factor = 0,1`; para VACUUM, `50` y `0,2`) y **solo actúa si el diagnóstico lo justifica**. Muestra `last_vacuum`, `last_autovacuum`, `last_analyze` y `last_autoanalyze` antes y después. Se ejecutó sobre la base restaurada y sobre la principal.

| TABLA (BASE RESTAURADA) | FILAS | CAMBIOS / UMBRAL ANALYZE | LAST_AUTOVACUUM / LAST_AUTOANALYZE | LAST_ANALYZE ANTES → DESPUÉS | DECISIÓN |
|---|---|---|---|---|---|
| detalle_pedido | 499.263 | 0 / 49.976 | 23:12:19 / 23:12:19 | — → — | sin acción |
| pedido | 200.005 | 0 / 20.051 | 23:12:20 / 23:12:20 | — → — | sin acción |
| producto | 50.011 | 0 / 5.051 | 23:12:20 / 23:12:20 | — → — | sin acción |
| cliente | 20.005 | 0 / 2.051 | 23:12:20 / 23:12:21 | — → — | sin acción |
| **categoria** | 5 | 5 / 50,5 | — / — | **— → 23:12:47** | ANALYZE |
| **usuario** | 3 | 3 / 50,3 | — / — | **— → 23:12:47** | ANALYZE |

- **Por qué ANALYZE solo en `categoria` y `usuario`.** Tras la restauración, autovacuum analizó solo las cuatro tablas grandes, entre las 23:12:19 y las 23:12:21. Las dos chicas quedaron **sin estadísticas**: con 5 y 3 filas, sus cambios nunca alcanzan el umbral de autoanalyze (≈ 50), así que autovacuum no las analizaría nunca y el planificador estimaría a ciegas. Por eso, además del umbral, el script considera "sin estadísticas" como motivo, y analiza solo esas dos.
- **En `foodstore`:** las seis tablas tienen estadísticas (último `ANALYZE` el 28/10) y 0 cambios pendientes. Decisión: *sin acción, estadísticas al día*.
- **Por qué no VACUUM:** en ambas bases las tuplas muertas son 0 a 2 por tabla, contra umbrales de 51 a 99.903. Autovacuum alcanza y un `VACUUM` manual no aportaría nada.
- **Por qué no VACUUM FULL:** reescribe la tabla completa y la bloquea con `ACCESS EXCLUSIVE`, sin lecturas ni escrituras mientras dura. Sirve para recuperar espacio tras borrados masivos, no como mantenimiento periódico, y la consigna lo excluye.

Antecedente del 28/10: `foodstore` se creó con `createdb -T bd2_tp3`, que copia los datos pero no los contadores de actividad. `health_check.sql` mostró `n_live_tup = 0` en tablas con 499.263 filas, y esa observación justificó el primer `ANALYZE` (evidencia `maintenance_salida.txt`).

## 6 Problemas reales encontrados

**Fin de línea de Windows (28/10)**
`psql` devolvía cada fila terminada en `\r\n` y la tabla de validación se imprimió superpuesta (`restore_test_intento1_crlf.txt`). Se corrigió con `tr -d '\r'`, y `.gitattributes` fija los `.sh` en LF para que no se rompan al clonarlos en Windows.

**Tablas chicas sin estadísticas (07/11)**
En la base restaurada, `categoria` y `usuario` quedaron sin `last_analyze` ni `last_autoanalyze`. Autovacuum no las cubre por debajo del umbral. Se detectó con el diagnóstico y se resolvió con un `ANALYZE` limitado a ellas.

**Mensajes de error con acentos rotos**
El detalle de los `FAIL` en el log aparece como `fall\ufffd la conexi\ufffdn`: Windows y PostgreSQL usan codificaciones distintas. El resultado (FAIL, paso y código) se lee bien. Queda como mejora pendiente.

**La huella no contaba `usuario` (07/11)**
La primera versión de `conteo_referencia.sql` contaba 5 de las 6 tablas: una restauración que perdiera `usuario` habría pasado. Lo detectó la revisión de OpenCode y se corrigió (huella de 11 claves).

## 7 Uso de IA y control humano

| HERRAMIENTA | QUÉ HIZO | CÓMO SE CONTROLÓ |
|---|---|---|
| **Claude** | Primera versión de los scripts y de este informe, a partir de la consigna | Cada script se probó en mi PC antes de commitearlo, incluidos los caminos de error (base inexistente, borrado no autorizado) |
| **Kiro** | Reescribió `SPEC_U6.md` con los requisitos nuevos | Error detectado: indicó el formato del `.sha256` al revés (`sha256sum` escribe primero el hash). Se corrigió a mano (commit `2f8ea9c`). El "diff" que mostró lo armó él: se verificó con `git diff --stat` |
| **OpenCode** | Dos revisiones contra la spec (`revision_opencode.md`, 28/10, y `revision_opencode_v2.md`, 07/11) e implementación de los cambios aceptados | De la v2: **7 hallazgos aceptados, 1 rechazado y el resto pospuestos**, con motivo y verificación en `decisiones_revision.md`. Se le exigió la salida literal de `git --no-pager diff` y se revisó línea por línea antes de ejecutar |

**Constancias concretas de control humano:**

- **Autorización de cada borrado.** `restore_test.sh` no borra `foodstore_restore_test` hasta que se escribe su nombre. Cada autorización queda en `logs/restore_test.log` con usuario, origen y hora; por ejemplo, `23:12:10 | AUTORIZADO | usuario=beluu | origen=interactivo`. Una prueba sin autorizar (22:32:33) quedó registrada como `FAIL` y no tocó ninguna base.
- **Errores de la IA detectados y documentados.** En la primera revisión, el diff de OpenCode no coincidía con el archivo real (`rto_ms` en lugar de `rto_s`, comprobado con `grep`). En la segunda, OpenCode afirmó que había archivos sin commitear que ya estaban commiteados, y dejó un archivo vacío en la raíz que se borró. A Kiro se le corrigió el formato del `.sha256`.
- **Decisiones propias.** Se rechazó cambiar la regla "sin estadísticas" (sección 5) y se pospusieron mejoras de bajo impacto. Cada decisión está en `decisiones_revision.md`, y cada cambio en un commit separado de la rama `belen-tp6-correccion`.

## 8 Mejoras para la próxima ejecución y conclusión

- **Roles y permisos:** respaldar los objetos globales con `pg_dumpall --globals-only`, restaurarlos antes del `.dump` y quitar `--no-owner --no-privileges`, para recuperar también dueños, roles y `GRANT`.
- **Programar el respaldo** diario en el Programador de tareas de Windows con retención de 7 días, para que el RPO de 24 h no dependa de correrlo a mano (con ~9,4 MB por respaldo, sin rotación serían ~3,4 GB al año).
- **Ampliar la huella** con la cantidad de triggers y funciones, porque las reglas de producto activo y stock suficiente viven en triggers sobre `detalle_pedido`. Y normalizar la codificación de los mensajes de error del log.

**Conclusión.** Food Store se recuperó en `foodstore_restore_test` con la huella completa idéntica al respaldo (filas, total facturado y estructura) y los 8 indicadores de integridad en 0, en 4,21 s, frente a una meta de 15 min. El procedimiento verifica el respaldo cuando se genera y antes de usarlo, no borra nada sin autorización escrita y solo hace mantenimiento cuando los datos lo justifican.

### Evidencias en el repositorio

- Carpeta del TP6: github.com/belcalvo93/base-datos-2/tree/main/mantenimiento (cómo reproducirlo en `mantenimiento/README.md`)
- Simulacro final: `mantenimiento/evidencias/restore_test_20261107_231154.txt` · no autorizado: `…/restore_test_20261107_223233.txt`
- Mantenimiento: `mantenimiento/evidencias/maintenance_v4_restore_test.txt` y `…/maintenance_v4_foodstore.txt`
- Logs: `mantenimiento/logs/backup_foodstore.log` (PASS/FAIL, bytes, SHA256) y `mantenimiento/logs/restore_test.log` (autorizaciones y resultados)
- Ensayos del grupo: `mantenimiento/evidencias/hernan/`, `elias/`, `bruno/` · Revisiones y decisiones: `mantenimiento/revision_opencode.md`, `revision_opencode_v2.md`, `decisiones_revision.md`